

计算机辅助制造实践

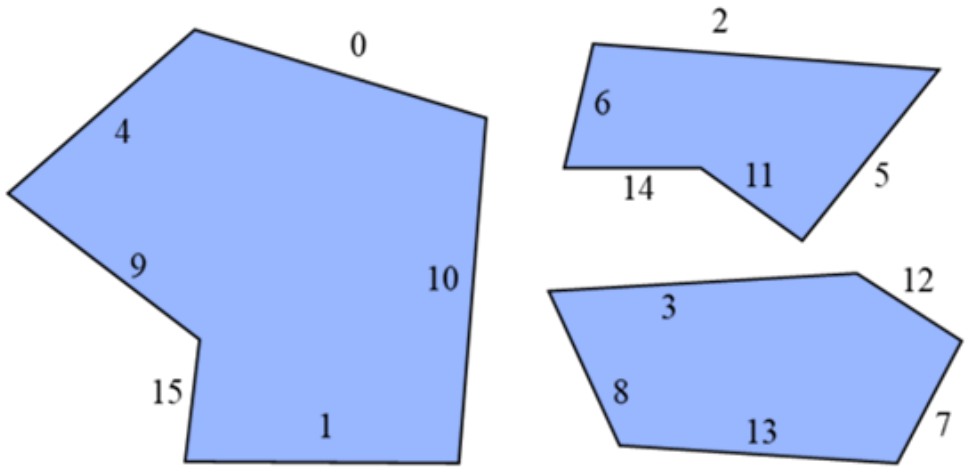
——Python实现三维打印路径规划

浙江大学机械工程学院 林志伟

玉泉教1-215　zjtzhylin1986@163.com

第6章 截交线段拼接计算优化

- 暴力拼接存在的问题
- 链接点数据结构
- 字典序排序法拼接优化
- 字典查询法拼接优化
- 算法测试比较

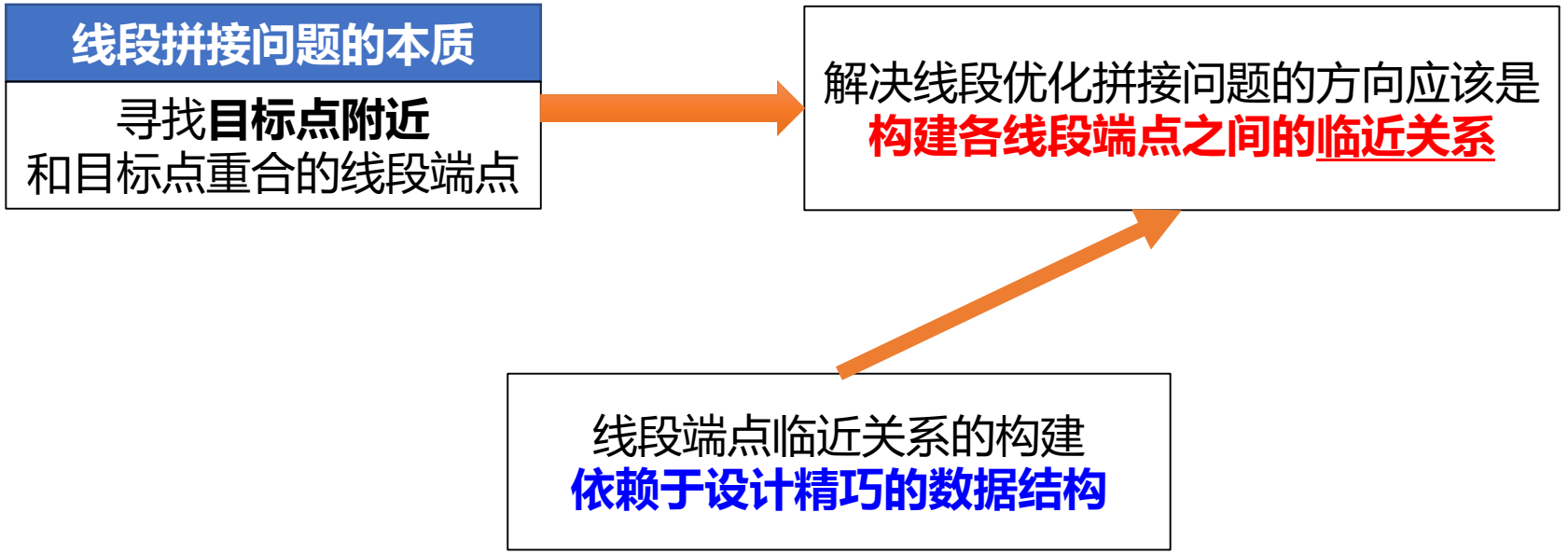


1. 暴力拼接存在的问题

- **截交线段拼接问题可以抽象为**：对同一平面上N条**散乱无序**的线段序列，将其**首尾相连**成M条多段线。
  - 这里M ≤ N，如果M = 1，则所有N条线段都将连接到同一条多段线上，如果M = N，则N条线段中无两条线段首尾坐标重合。
  - 对三维打印这一具体应用来说，最终拼接得到的多段线应该还需满足**首尾相接**的要求，即它们应该是**闭合**的。
  - 拼接的难点**是由STL模型中三角面片的无序性造成的**。
- **暴力拼接基本原理：**对于当前拼接过程中的多段线，遍历线段序列，从头开始搜索能和当前多段线端点（起点或终点）重合的那条线段，如果能够找到线段，只需将该线段拼接到多段线上，如果找不到线段，则对当前多段线来说，拼接过程结束。
- **暴力拼接的问题：**暴力拼接法在最优的情况下，仍需要经历n(n+1)/2次搜索和坐标比对，其中n为线段序列中的线段数量，即暴力截交法的时间复杂度为$O(n^2)$。
  - 在实际测试中，对monk模型（面片数为336974），当切片层厚为0.2mm，切片层数为877层时，使用扫描平面法（sweep）所需的截交时间约为21秒，得到的各层截交线段总数为866540条，平均每层线段数约988条。对这些层上的截交线段，使用暴力法进行线段拼接的总时间约为975秒，**超过16分钟**，平均每层拼接时间超1.1秒。
- **目标：**设计高效、稳定的散乱线段拼接算法。

2. 链接点数据结构

- 待拼接的截交线段以**Segment**数据结构保存了两端点坐标（A、B）。
- Segment结构过于简单、基础，仅仅依靠Segment结构，无法对线段拼接算法做出进一步优化。



- **链接点LinkPoint**：既可以保存一个**点**，也可以保存一条**线段**！

2.链接点数据结构

- **LinkPoint**本质上是一个点，它的具体定义如下（写在LinkPoint.py文件中）：

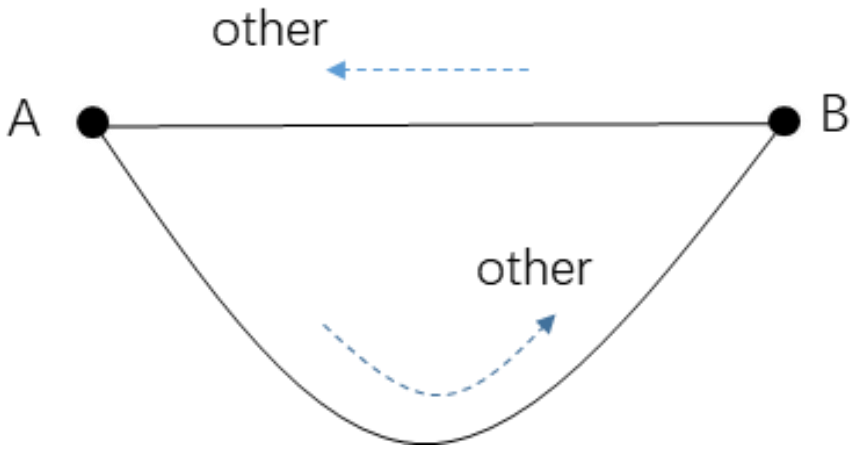
LinkPoint.py | **链接点数据结构** | **6-1**

```
1   from GeomBase import Point3D                    # 导入Point3D类
2   class LinkPoint():                              # 链接点类，LinkPoint
3       def __init__(self, pnt3d, digits = 7):      # 根据Point3D对象初始化
4           self.x = round(pnt3d.x, digits)         # 点的x、y、z坐标
5           self.y = round(pnt3d.y, digits)
6           self.z = round(pnt3d.z, digits)
7           self.other = None                       # 指针，指向线段另一个端点
8           self.used = False                       # 点是否已经被使用
9           self.index = 0                          # 点在列表中的序号
10      def __str__(self):                          # 转字符串函数，方便调试
11          return 'LinkPoint: used: %s\nself (%s, %s, %s)\nother (%s, %s, %s)\nlinked to: %s' \
12              % (self.used, self.x, self.y, self.z, self.other.x, self.other.y, self.other.z, self.other.index)
13      def toPoint3D(self):                        # LinkPoint转化为Point3D对象
14          return Point3D(self.x, self.y, self.z)
```

2.链接点数据结构

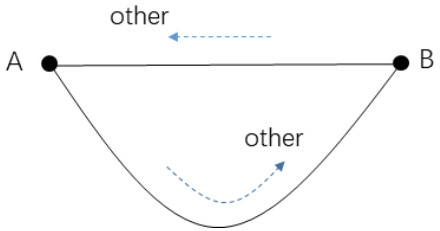
https://zhuanlan.zhihu.com/p/351034359

- LinkPoint的**__init__**函数使用Point3D的对象来给x、y、z坐标赋值，并通过digits变量和Python的round函数来控制连接点坐标的小数位数。这里默认设置的digits为7，即通过初始化函数得到连接点坐标的**小数位数为7**。
  - 设立digits变量和计算机在浮点运算中的截断误差有关，截断误差可能导致本应重合的两个端点不重合，比如理论结果应该是1.0，然而实际计算可能可到类似0.99999999或1.00000001这样的结果。
- **重要属性other**，该属性为一指针，指向另一个LinkPoint类型的对象。
  - 对线段的A、B两个端点，如果LinkPoint本身的 **x、y、z** 坐标储存了A这个端点坐标，那么**other**将表示B这个端点。



链接点具备的点和线段双重特性

2.链接点数据结构



- LinkPoint属性**other**赋值：

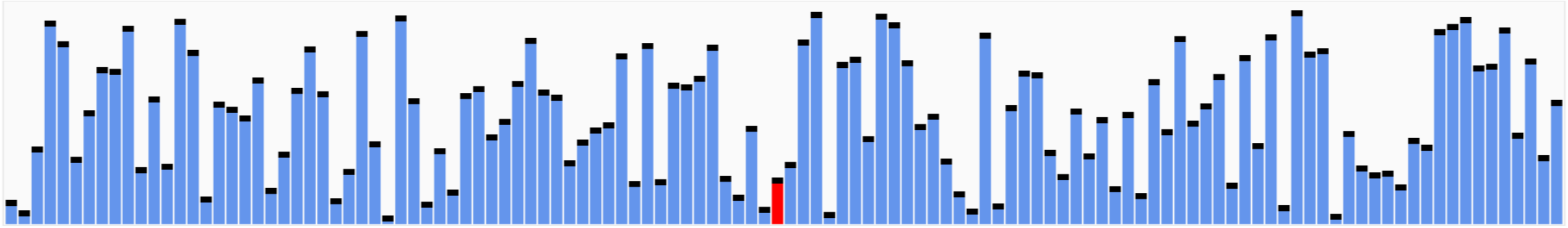
	Segment和LinkPoint数据相互转化	6-2
1	lp1 = LinkPoint(A)	#A、B为线段Segment的两端点
2	lp2 = LinkPoint(B)	#A、B为Point3D类型对象
3	lp1.other = lp2	#链接到lp2，而非B
4	lp2.other = lp1	#链接到lp1，而非A

- 由以上代码可知：
  - LinkPoint具备了线段两端点信息，因此，它也可以用来表示一条线段。
  - 将一条线段转化成链接点时，会得到两个不同的链接点和两条完全重合的线段。
- 在LinkPoint类中，还定义了**used**和**index**两个属性。
  - used属性为布尔型变量，默认值为False，表示当前链接点在线段拼接过程中是否已经被使用过，显然，如果一个点已经被使用过，它就不应该被使用第二次，否则得到的拼接轮廓将产生拓扑错误。
  - index属性用来表示当前点在列表中的序号。一般情况下，在用列表存储数组元素时，可以通过列表和已知元素序号很快查询到对应元素，但是当只有元素本身时，却很难定位它在列表中的序号（只能通过遍历比对）。借助元素中保存的index属性，便能很快定位元素，并找出和它具有相邻序号的其他元素。

3.字典序排序法拼接优化

- 在现实生活中，可能经常需要处理那些包含**大量同类型元素**的优化问题。针对这些问题，暴力遍历所有元素需要消耗大量的运算时间，不太可取。
- 在想到更好的办法之前，先**对这些元素进行排序**可能会是一个不错的选择，这会激发进一步解决问题的灵感。
  - 在前面处理STL模型截交问题时已经做过了尝试，通过对三角面片最低点进行排序，实现了扫描平面快速截交算法。
  - 在处理同一平面上大量截交线段的拼接问题时，也许也应该先对这些线段进行排序。进一步地，当把线段转化为LinkPoint对象后，也许可以考虑先**对链接点进行排序**。

——犹豫不决先排序！



3.字典序排序法拼接优化

- **字典中单词的排列规则**

- 如果单词中**只有一个字母**，那么这些单词将按照 a~z 的顺序排列，比如a在b的前面（记为a < b），b在c的前面（记为b < c）等。
- 如果单词由**一个或两个字母**组成，那么这些单词先按照第一个字母排序，如果第一个字母相等，则继续按照第二个字母排序，比如ab < b，ab < ba，ab < ac等。
- 同理，如果单词**由一个、两个或三个字母**组成，那么这些单词先按照第一个字母排序，如果第一个字母相等，则按照第二个字母排序，如果第二个字母也相等，则按照第三个字母排序，比如abc < bc，a < bcd，abc < acd等。
- 以此类推，可以按照这种规则对字典中所有的单词排序，**每个单词也总能在字典中找到一个合适的位置**。



3.字典序排序法拼接优化

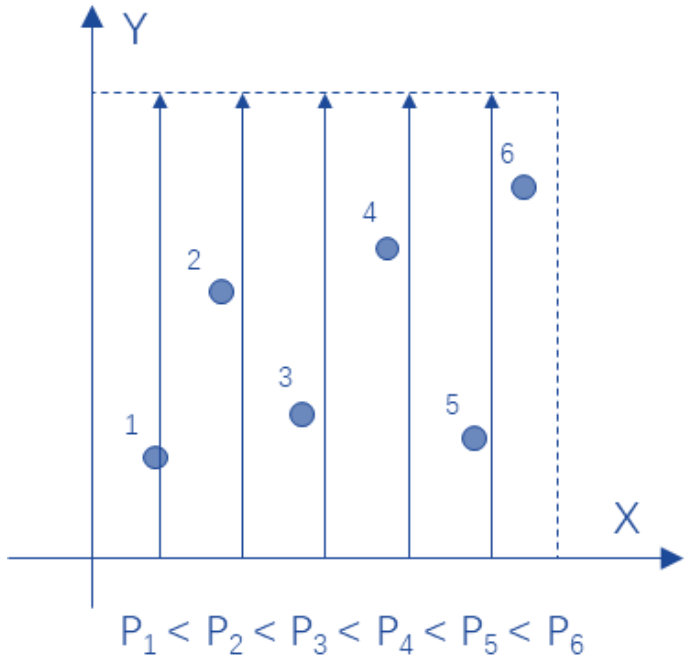
- **点的字典序**
- **方法1：坐标转字符串**
  - 把点的坐标数值转化为字符串，然后对字符串进行排序。

> **例**：对点$P_1(1.1, 2.2, 3.4)$和$P_2(1.1, 3.4, 4.5)$，可以两个点的坐标分别格式化为“**112234**”和“**113445**”字符串，然后对两个字符串按ASCII码顺序的字典序排序，最终可以得到$P_1 < P_2$。

  - **存在的问题**：**没有任何几何意义！**且点坐标值的波动对排序结果影响极大。比如P_2坐标变为(1.10, 3.4, 4.5)，这样的变化根本不会改变P_2的几何意义，但此时排序的结果将变为$P_2 < P_1$，这是不应该发生的。

3.字典序排序法拼接优化

- **点的字典序**
- **方法2：各维度坐标分别比较**
  - 对任意两个点$P_1(x_1, y_1, z_1)$和$P_2(x_2, y_2, z_2)$
    - 首先比较两个点的X坐标：如果$x_1 < x_2$，那么$P_1 < P_2$。
    - 如果$x_1 = x_2$，再比较两个点的Y坐标：如果$y_1 < y_2$，那么$P_1 < P_2$。
    - 如果$y_1 = y_2$，再比较两个点的Z坐标：如果$z_1 < z_2$，那么$P_1 < P_2$。
  - 通过这种比较方式，即便P_1、P_2坐标发生微小扰动，也不会影响比较结果。
  - 这种排序的几何意义是非常明确的，在二维情况下，**左下方**的点总是**小于右上方**的点。



3.字典序排序法拼接优化

- **点的字典序**
- **定义点的“小于号”比较函数**
  - **cmp_pntSmaller**函数（全局函数）：实现针对两个LinkPoint对象的小于比较，将其写在LinkSegs_dorder.py文件中。

LinkSegs_dorder.py	定义两个链接点（LinkPoint）的小于比较函数	6-3

```
1   ……                                                  # 导入必要库
2   def cmp_pntSmaller(lp1, lp2):                       # 定义LinkPoint的比较函数
3       if (lp1.x < lp2.x):                             # 首先比较x值
4           return -1                                   # 成立，则返回-1，表示lp1 < lp2
5       elif (lp1.x == lp2.x and lp1.y < lp2.y):        # 再比较y值
6           return -1
7       elif (lp1.x == lp2.x and lp1.y == lp2.y and lp1.z < lp2.z):      # 在比较z值
8           return -1
9       elif (lp1.x == lp2.x and lp1.y == lp2.y and lp1.z == lp2.z):     # 如果都相等，返回0
10          return 0
11      else:                                           # 否则返回1，表示lp1 > lp2
12          return 1
```

  - cmp_pntSmaller函数输入参数为lp1和lp2，均为LinkPoint类型的对象。针对函数返回，定了3种情况：
    - 如果lp1 < lp2，则返回–1；
    - 如果lp1 = lp2，则返回0；
    - 如果lp1 > lp2，则返回1。

3.字典序排序法拼接优化

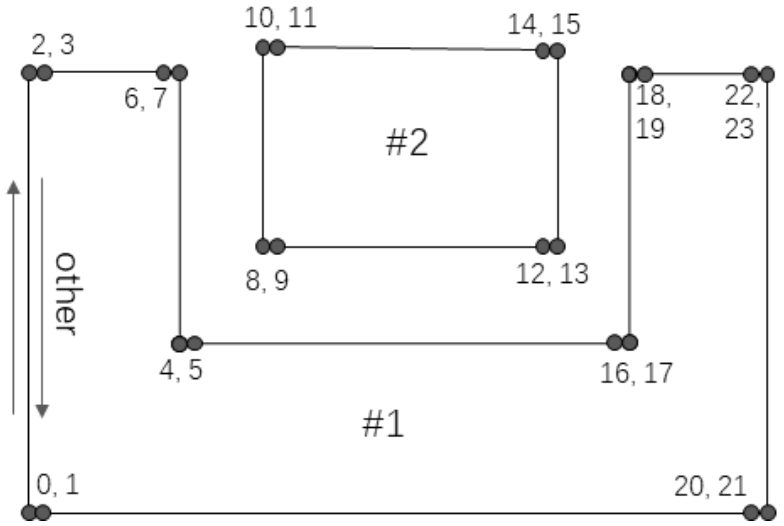
- **点的字典序**
- **链接点的字典序排序**
  - 之所以将cmp_pntSmaller函数的返回分为3种情况，而不是简单的True或False，是因为要把这个函数作为回调函数，输入到Python的sort函数中，而sort函数对回调函数的规定是必须返回这3种比较结果。事实上，其他语言（如C++）的排序函数也有类似规定。
  - 在这个函数的基础上，只要通过下列代码，就能完成对链接点序列的排序：

```
1 | lpnts.sort(key = functools.cmp_to_key(cmp_pntSmaller))
```

其中lpnts为用于存放链接点的列表，cmp_to_key可将定义的比较函数转化为sort函数中的key参数。但使用cmp_to_key需要手动导入functools库。

3.字典序排序法拼接优化

- **字典序拼接原理**
- 将平面上M条散乱线段转化为2M个LinkPoint对象，暂存在lpnts列表中，并按字典序对lpnts中的元素进行排序。平面上点的分布如图所示：

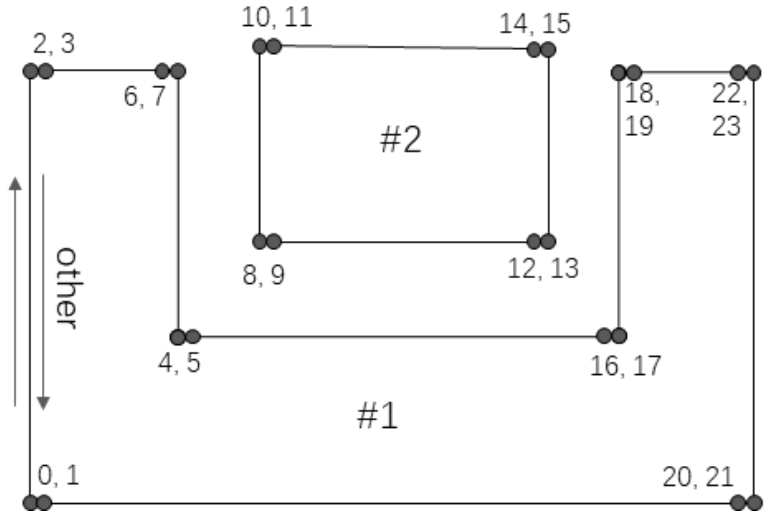


- **寻找这些点在平面上的分布有什么规律?**

3.字典序排序法拼接优化

排序前：2,4,5,3,3,2,5,3….
排序后：2,2,3,3,4,4,5,5….

- **字典序拼接原理**
- 点的分布规律：**序号间隔相邻的点是重合的**。



例如：

0号点和1号点重合，

2号点和3号点重合，

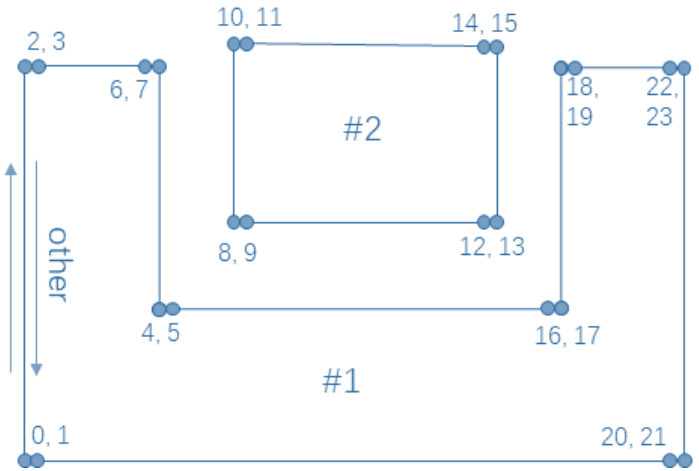
4号点和5号点重合，

以此类推……

- **这个规律是合情合理的**：
  - 首先，每对重合点就是原来相邻两条线段的公共端点，它们本应重合；
  - 其次，经过字典序排序之后，**坐标相同的点在列表lpnts中必然会被排到相邻的位置上**，即它们的序号必然相邻。

3.字典序排序法拼接优化

- **字典序拼接原理：拼接过程讲解**



- 首先，从24个点中选出任意一个点作为起点。为了不是一般性，选择3号点作为起始点。
- 因为这里的点都是LinkPoint类型的点，而3号、6号点本来就在同一条线段上，因此，从3号点的other属性便能立马定位6号点。
- 然后再从6号点出发，根据刚才发现的规律，和6号点重合的点在lpnts中的序号必然和6相邻，即要么5号点和6号点相邻，要么7号点和6号点相邻。
- 此时可以对5号、7号点做进一步检测，看哪个点到6号点的距离逼近0，哪个点就是6号点的重合点。在图中，显然，7号点为6号点的重合点。
- 接下来再循着7号点继续往下走，定位7号点的other点，即4号点，然后再从相邻序号中找出4号点的重合点5号点……

✓ **以此类推，依次找到16、17、19、18、…、1、0、2号点。当绕了一圈重新回到2号点时，发现走过的轮廓已经闭合，这时就可以结束当前寻找步骤。**

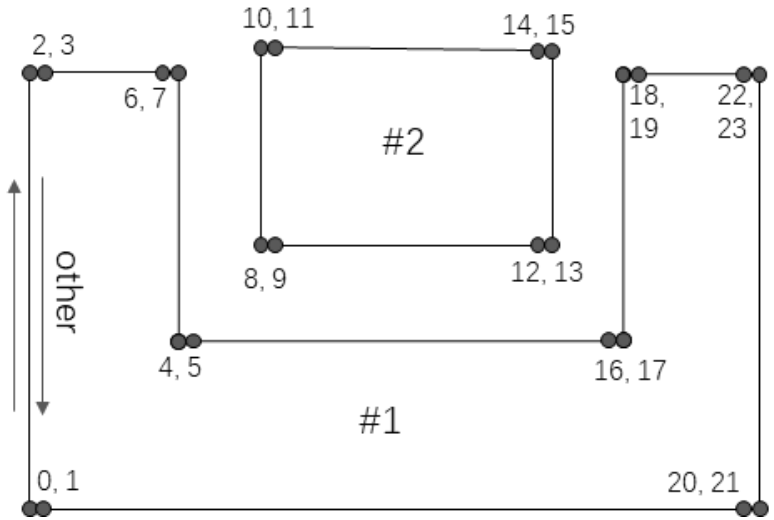
✓ **然后在找出一个没有用过的点（used 为 False），开始新一轮拼接……**

3.字典序排序法拼接优化

- **字典序拼接原理：index属性**

- 从3号点通过other定位到6号点后，在lpnts中寻找6号点的邻点5号点和7号点。**通过3号点的other定位的是点对象，而不是点的序号**，一般无法直接从对象获取它在列表中的序号。

- 幸运的是，在定义LinkPoint数据结构时，已经考虑到了这个情况。可以使用LinkPoint类中**index属性**标记当前点在列表中的序号。初始情况下，每个链接点的index属性值为0，index的赋值方法如下：

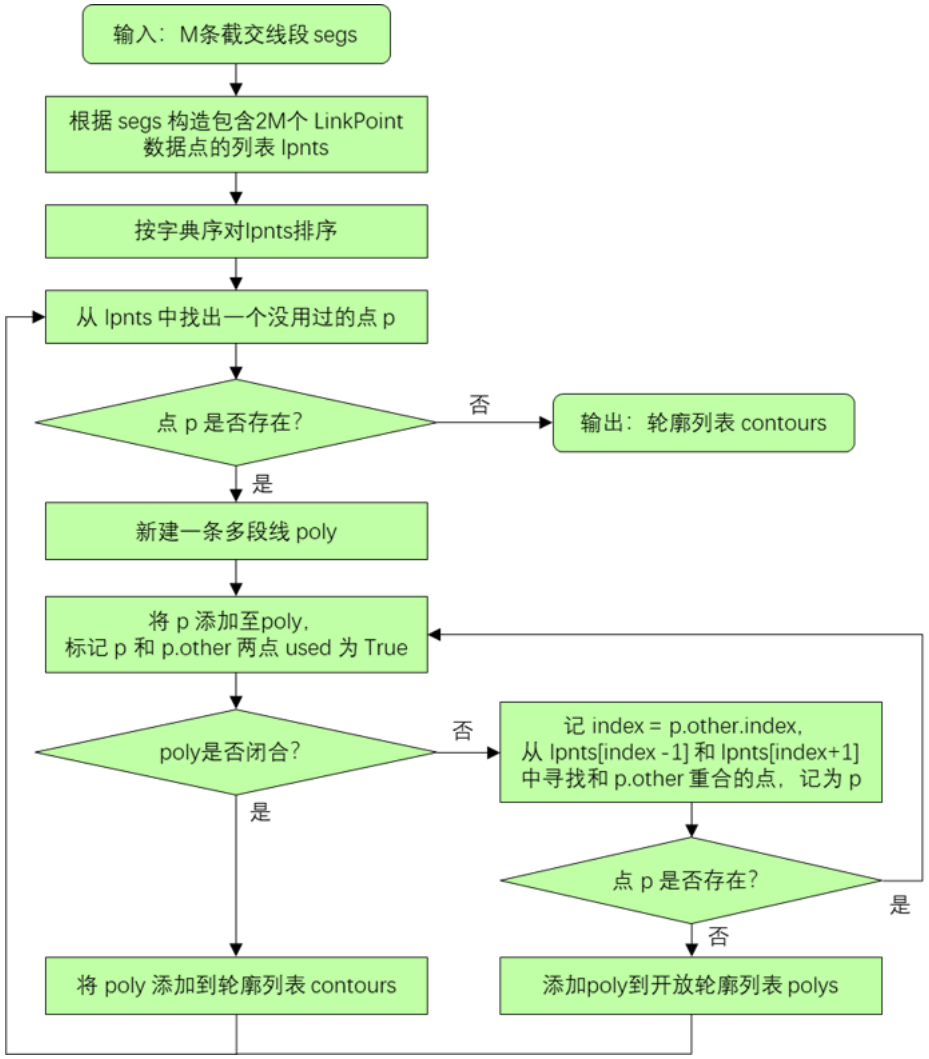



```
1   for i in range(len(lpnts)):          # 循环遍历lpnts
2       lpnts[i].index = i               # 对点的index属性赋值
```

- 借助index属性，可以从3号点的other属性获取other的index值，即它在lpnts列表中的序号（即6），然后通过这个序号找到和它相邻的序号（5和7）。

3.字典序排序拼接优化

- **算法流程**
- **输入**：**截交线段的列表segs**
- **输出**：**封闭轮廓的列表contours**
- 首先将Segment对象转化为LinkPoint对象，并将链接点保存到列表lpnts中。
- 对列表lpnts，按照字典序对其进行排序。
- 进入两层循环。其中第1层循环不断从lpnts中寻找没有用过的点p：
  - 如果点p不存在，即lpnts中所有点都已经被使用过，则跳出第1层循环，结束整个流程，输出轮廓列表contours；
  - 如果点p存在，则新建一条多段线poly，并进入第2层循环。
- 在第2层循环中，根据LinkPoint的other属性以及lpnts中点按字典序排序后重合点序号临近的特性，不断从lpnts寻找当前点的下一点添加到poly中，直至poly闭合或寻找下一点失败，跳出第2层循环并重新开始第1层循环。



3.字典序排序拼接优化

- **算法实现**

- 为了实现以上流程，专门封装了一个LinSegs_dorder类，类名中dorder表示dictionary order（字典序）的意思。

LinkSegs_dorder.py	LinkSegs_dorder类：封装基于字典序排序的线段拼接算法	6-4

```
1   ……                                          # 导入必要库
2   ……                                          # 略过cmp_pntSmaller函数
3   class LinkSegs_dorder:                      # 定义LinSegs_dorder类
4       def __init__(self, segs):               # 初始化，输入截交线段列表
5           self.segs = segs                    # 保存线段列表
6           self.contours = []                  # 最终得到的轮廓列表contours
7           self.polys = []                     # 用于保存非封闭的轮廓
8           self.link()                         # 调用拼接函数
9       def createLpList(self):                 # 根据输入线段构建链接点列表
10          lpnts = []                          # 链接点列表
11          for seg in self.segs:               # 遍历线段列表，构造链接点
12              lp1, lp2 = LinkPoint(seg.A), LinkPoint(seg.B)
13              lp1.other, lp2.other = lp2, lp1
14              lpnts.append(lp1)
15              lpnts.append(lp2)
16          lpnts.sort(key=functools.cmp_to_key(cmp_pntSmaller))    # 对点进行字典序排序
17          for i in range(len(lpnts)):         # 对点的index赋值
18              lpnts[i].index = i
19          return lpnts                        # 返回链接点列表
20      def findUnusedPnt(self, lpnts):         # 从lpnts中找出一个没用过的点
21      def link(self):                         # 定义线段字典序拼接核心函数
```

3.字典序排序拼接优化

- **算法实现**
- **link函数**是执行线段字典序拼接的核心函数，它基本还原了拼接流程：从数据点准备到第1层循环，第2层循环等。这里数据点准备是通过函数createLpList 函数实现的。
  - 在第1层循环中，通过findUnusedPnt函数从lpnts列表中找出一个没有用过的链接点，如果找不到这样的点，该函数返回–1，否则返回一个大于等于0的点序号。如果函数findUnusedPnt返回–1，则退出第1层循环。
  - 在第2层循环中，获得p.other的index值后，首先比较p.other和lpnts[index - 1]是否重合，这里将链接点转化为Point3D对象进行比较，比较函数是isCoincide而非isIdentical，这是一种偏安全的做法，可以增强拼接算法的鲁棒性。
- **link函数**退出条件有两个
  - 其一是正在生长的多线段poly闭合了，这种退出是正常的退出，得到的轮廓也是想要的封闭轮廓。
  - 其二是当前的p.other找不到序号相邻的重合点时退出，这种退出是不正常的退出，此时得到的多段线是非封闭的，这并不是我们想要的。

出现第二种情况的原因可能有两个：一是输入的STL模型本来就是非封闭的（有破洞或孔洞），二是STL模型中相邻面片间的缝隙过大，这其实都是模型本身的问题。

3.字典序排序拼接优化

- **全局函数**

- 在SliceAlgo模块中创建一个接口函数，函数名为**linkSegs_dorder**，用它封装LinkSegs_dorder类中的算法，输入为截交线段列表segs，输出为封闭轮廓列表contours，具体代码如下：

SliceAlgo.py	linkSegs_dorder：添加基于字典序排序的截交线段拼接算法	6-5

```
1  from LinkSegs_dorder import *      # 导入LinSegs_dorder模块
2  ……                                 # 忽略SliceAlgo其他导入模块及代码
3  def linkSegs_dorder(segs):                 # 定义函数
4      return LinkSegs_dorder(segs).contours  # 返回封闭contours
```

4.字典查询法拼接优化

- 以上介绍的**基于字典序排序**的截交线段优化拼接算法，其核心是依赖LinkPoint数据结构和点的字典序排序结果寻找线段之间的临近关系，它的理论时间复杂度为O(nlog(n))，其中n为截交线段数量。

- 下面将另辟蹊径，介绍一种在原理上完全不同的线段拼接算法。这种算法的基础依然是LinkPoint数据结构，它的本质仍然是构建和寻找线段之间的临近关系。

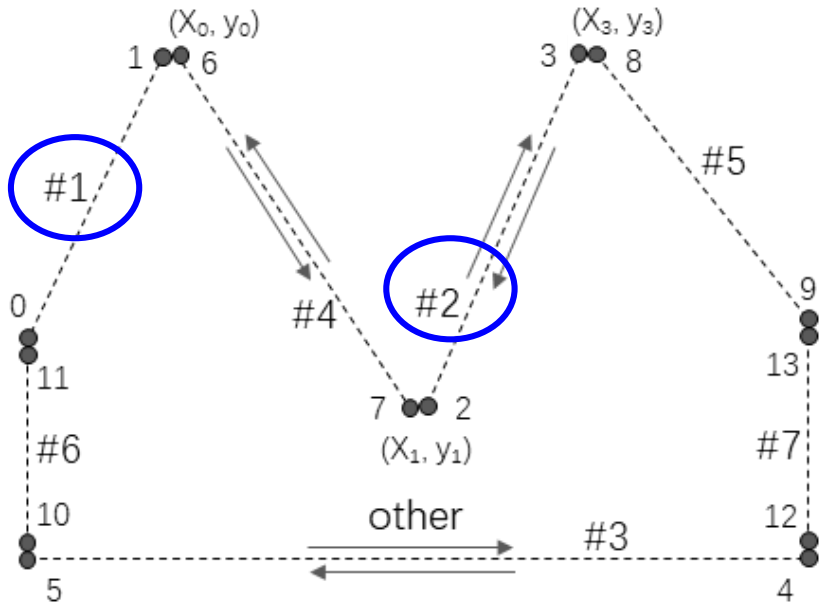
- 这种算法将所有转化得到的链接点放在一个字典对象中，依赖编程语言内置**字典类（dic，或者哈希表）**的快速查询功能定位临近点的信息。

- 注意，虽然两种方法都带“字典”，但两者是完全不同的概念，一个是指**字典序**，一个是指**字典类**。

4.字典查询法拼接优化

- **字典查询法拼接原理**

- 对平面上的M条散乱截交线段，可以把它们逐一拆分成一对对相互关联的链接点序列（不用排序操作），让它们以**最自然的状态排布在平面上**，如图所示。
- 此时，任意两个重合点的序号不再相邻，它们的差值不再为1，而是一个随机数。显然，在这种情况下，基于字典序的线段拼接方法将失效。

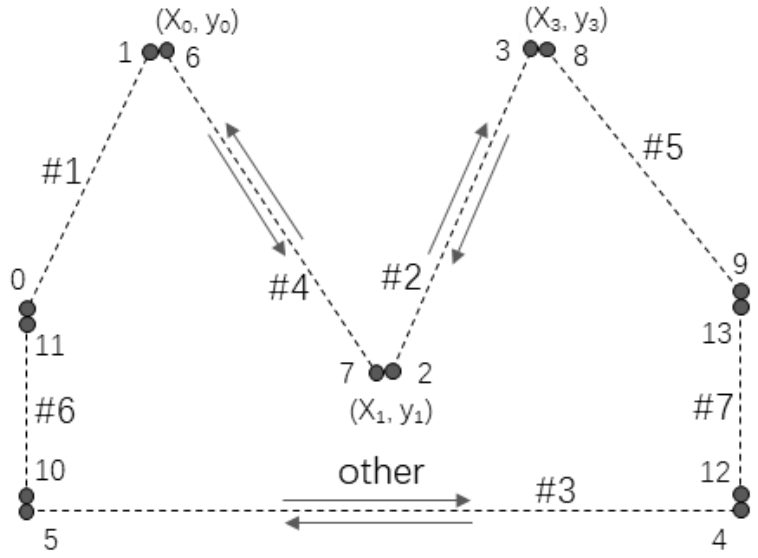


4.字典查询法拼接优化

- **字典查询法拼接原理**

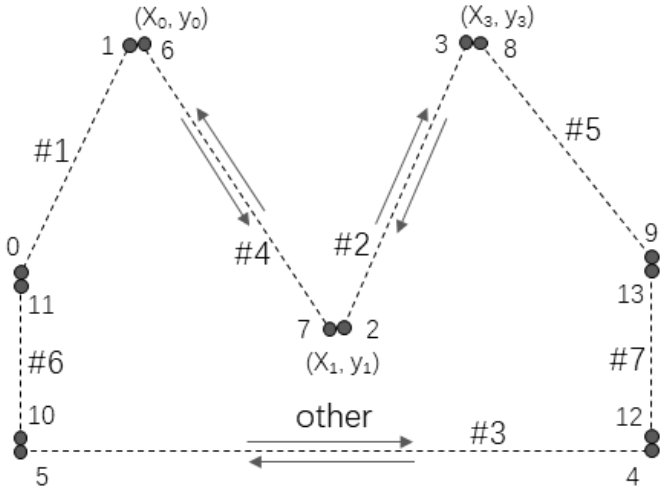
- 对1号和4号线段，这两条线段是相邻相接的，它们的端点分别为**0号、1号**和**6号、7号**链接点，其中1号点和6号点为重合点。由0号点是很容易定位到1号点的，只要通过0号点的other属性即可。**问题是：如何从1号点快速定位到6号点呢?**

- 比较1号点和6号点，它们俩唯一的相同之处就是**坐标**。如何通过它们共同的坐标将1号点和6号点相互绑定起来呢？（通过坐标搜索到两个点）



4.字典查询法拼接优化

- **字典查询法拼接原理**
  - **字典结构**：键值对，**键**就是两点的共同坐标，**值**就是被绑定的两个点对象。
  - 用Python语法描述，1号点和6号点在字典中的保存形式为{(x_0, y_0): [lp1, lp6]}。
    - (x_0, y_0)是由1号点和6号点共同坐标组成的元组，它是字典中的键；
    - lp1和lp6分别为1号和6号链接点，[lp1, lp6]为保存了两链接点的列表，它是字典中的值。
  - 以上字典只有1个元素，如果该字典保存了多个元素，可以通过坐标(x_0, y_0)查询得到列表[lp1, lp6]。

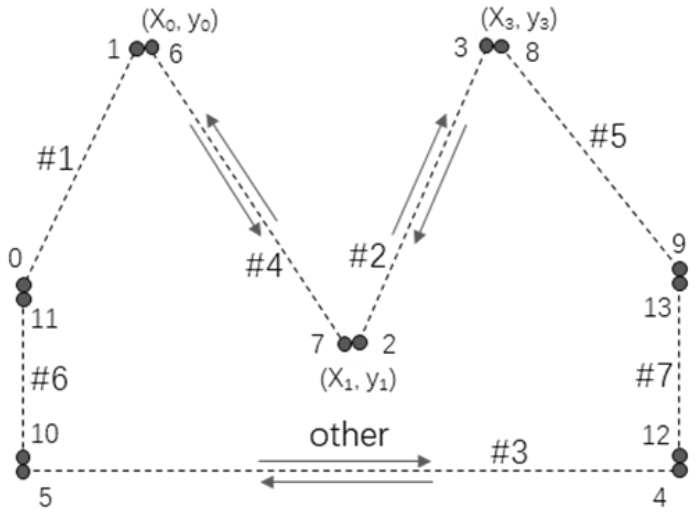


注意：
从字典中通过键来查询值的速度很快，具体地，查询的时间复杂度为**O(1)**，即常数时间完成。

4.字典查询法拼接优化

• 字典查询法拼接原理

- 结合{(x_0, y_0): [lp1, lp6]}字典来分析一下**如何从1号点定位到6号点。**
  - 从0号点开始，根据0号点的other定位到1号点；
  - 然后把1号点的X和Y坐标提取出来，构成(x, y)元组；
  - 根据这个元组从字典中查询得到对应的 [lp1, lp6] 列表，但这个列表中两个元素是无序存储的；
  - 接下来通过比对1号点和 [lp1, lp6] 中元素的指针地址（Python中为id）来找出6号点。
- 通过以上步骤，可从1号点成功定位到了6号点。
- 同理，假如字典中元素变成{(x_0, y_0): [lp1, lp6], (x_1, y_1): [lp7, lp2]}，此时可以通过0号点开始依次找到1号点，6号点，7号点，2号点和3号点。
- 如果把平面上所有重合点都按照以上形式放到字典中，就可以非常快速地将一条条轮廓线找出来。



4.字典查询法拼接优化

- **链接点字典构建**
- **对平面上的M条散乱线段segs，将其转化为字典dic存储的步骤如下：**

  步骤1. 遍历列表segs，从segs中取出当前线段，记为seg；

  步骤2. 根据seg构建两个链接点lp1和lp2，并对两点中的other属性相互赋值；

  步骤3. 将lp1添加到dic中以元组(lp1.x, lp1.y)为键的点列表中；

  步骤4. 将lp2添加到dic中以元组(lp2.x, lp2.y)为键的点列表中；

  步骤5. 跳至步骤1，直至遍历结束。

4.字典查询法拼接优化

• 链接点字典构建

- 将以上步骤封装成一个**createLpDic**函数，该函数写在LinkSegs_dlook类中，对应的文件为LinkSegs_dlook.py。注意：LinkSegs_dlook是一个和LinkSegs_dorder类相互平行的类，它们的初始化函数、属性以及类结构也完全一致。

LinkSegs_dlook.py | **在LinkSegs_dlook类中定义构建字典函数constructDic** | **6-6**

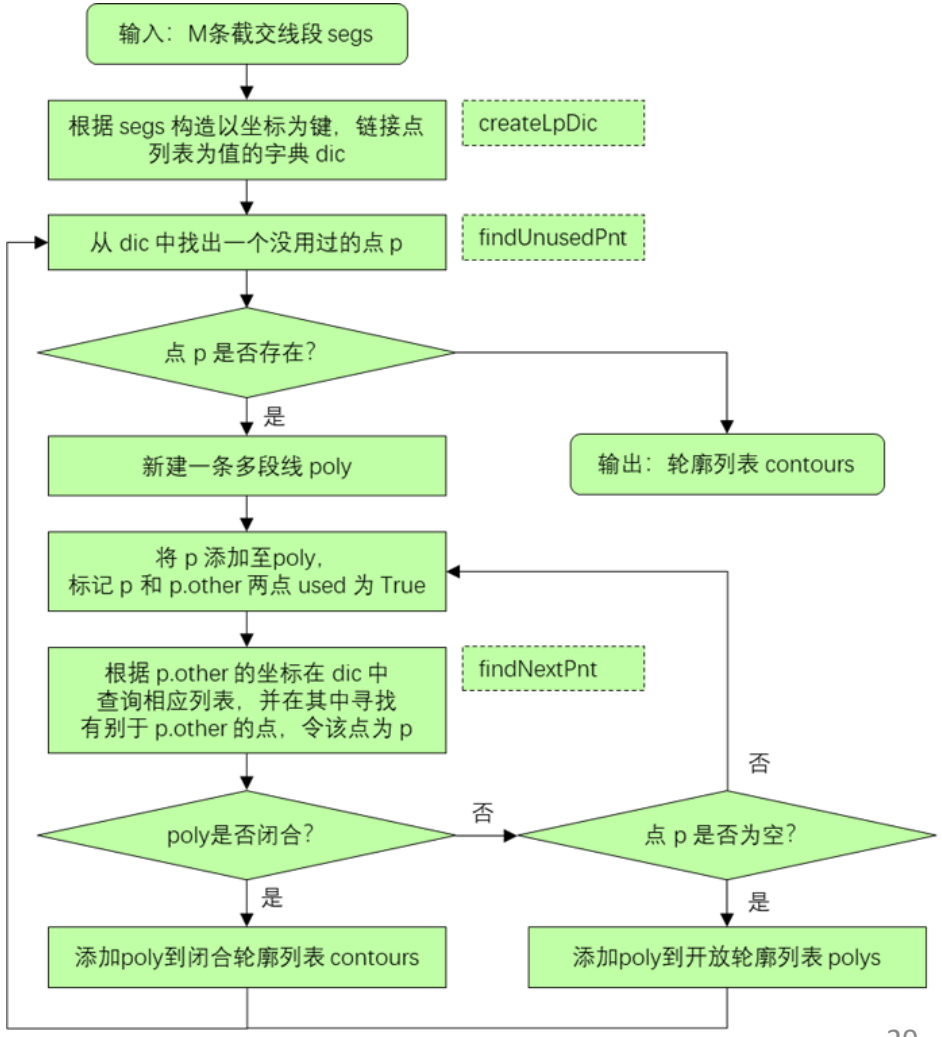
```
1   ……                                          # 导入必要库
2   class LinkSegs_dlook:                       # 类定义
3       def __init__(self, segs):               # 初始化函数，输入线段列表segs
4           self.segs = segs                    # 保存线段列表
5           self.contours = []                  # 保存封闭轮廓
6           self.polys = []                     # 保存非封闭轮廓
7           self.link()                         # 调用拼接函数
8       def link(self): pass                    # 字典查询法拼接函数，略
9       def createLpDic(self):                  # 构建字典函数
10          dic = {}                            # 字典初始化
11          for seg in self.segs:               # 遍历segs
12              lp1, lp2 = LinkPoint(seg.A), LinkPoint(seg.B)   # 构建两个LinkPoint数据点
13              lp1.other, lp2.other = lp2, lp1 # 点的other属性相互赋值
14              if (lp1.x, lp1.y) not in dic.keys():    # 判断lp1坐标是否在字典的键中
15                  dic[(lp1.x, lp1.y)] = []    # 如果不在则对该键新建列表
16              dic[(lp1.x, lp1.y)].append(lp1) # 对该键对应的列表添加lp1
17              if (lp2.x, lp2.y) not in dic.keys():    # 对lp2执行相同的操作
18                  dic[(lp2.x, lp2.y)] = []
19              dic[(lp2.x, lp2.y)].append(lp2)
20          return dic                          # 返回字典
```

- 在正常情况下，如果输入的线段数为M，则字典中键的个数也为M，每个键对应的列表存放两个链接点。

4.字典查询法拼接优化

- **拼接算法流程**

- **输入**：截交线段的列表segs
- **输出**：封闭轮廓的列表contours
- 首先从dic中寻找一个从来没有被使用过的点p。因为仅在字典的值中保存了链接点对象，因此遍历字典的值（而非键），从中寻找符合要求的点p。
- 如果不能找到这样的点，那么退出循环，输出轮廓列表contours，整个流程到此结束；但如果能够找到没被使用过的点p，则新建一条多段线poly，用于收集链接点。
- 将点p作为当前点添加到多段线poly中，并设置p和p.other两点的used属性为True，即两点已经被使用。
- 然后根据p.other的坐标在dic中查询相应的链接点列表，并在该列表中选择有别于p.other的点作为p的更新目标，即设置该点为新的点p（此时p也有可能为空，即查询失败）。
- 判断此时poly是否闭合，如果闭合则将poly添加到闭合轮廓列表contours中，并跳出循环；如果不闭合则进一步判断p是否为空，如果为空则将poly添加到开放轮廓列表polys中，并跳出循环。在以上两个条件均不成立时，则继续循环，不断从dic中收集点添加到poly中



4.字典查询法拼接优化

- **拼接算法实现**

LinkSegs_dlook.py	LinkSegs_dlook类：封装基于字典查询的截交线段优化拼接方法	6-7

```
1   ……                                          # 导入必要库
2   class LinkSegs_dlook:                       # 类定义
3       def __init__(self): pass                # 忽略之前已定义过的函数
4       def createLpDic(self): pass
5       def findUnusedPnt(self, dic):           # 从字典中寻找没用过的点
6           for pnts in dic.values():           # 遍历字典的值（values）
7               for pnt in pnts:                # 正常情况下，pnts包含2个元素
8                   if pnt.used == False:
9                       return pnt
10          return None                         # 没找到没用过的点，返回None
11      def findNextPnt(self, p, dic):          # 根据当前点p寻找下一个点
12          other = p.other
13          pnts = dic[(other.x, other.y)]      # 以点p的other坐标为键查询
14          difPnt = None                       # 不同于other的点
15          for pnt in pnts:
16              if pnt is not other:            # 指针（引用、id）比较
17                  difPnt = pnt
18          return difPnt                       # 返回找到的不同于other的点
19      def link(self):                         # 定义链接核心函数link
20          dic = self.createLpDic()            # 首先构建字典
21          while True:                         # 第1层循环
22              p = self.findUnusedPnt(dic)     # 寻找没用过的点p
23              if p is None: break             # 如果没找到p则退出第1层循环
24              poly = Polyline()               # 新建多段线poly
25              while True:                     # 第2层循环
26                  poly.addPoint(p.toPoint3D())# 将点p添加至poly
27                  p.used, p.other.used = True, True # 设置p和p.other为已用
28                  p = self.findNextPnt(p, dic)# 根据p和dic寻找下一个点
29                  if poly.isClosed():         # 如果poly闭合则将其添加至
30                      self.contours.append(poly) # 封闭轮廓列表contours中，并
31                      break                   # 退出第2层循环
32                  if p is None:               # 如果poly不闭合则将其添加至
33                      self.polys.append(poly) # 开放轮廓列表polys中，并
34                      break                   # 退出第2层循环
```

4.字典查询法拼接优化

- **拼接算法实现**
- 在以上代码中，从头到尾没用到过LinkPoint结构中的index属性，因此，**对字典查询法来说，LinkPoint的index属性是不必要的**。
- **findUnusedPnt**函数的作用就是在字典中寻找一个没有被使用过的点，它遍历的是字典的值（values）而非键，它的返回值为点本身（对象本身，或者叫指针或引用）或None，如果返回None表明在字典中所有点都已经被使用过。
  - 注意，这里这个函数的返回值类型和LinkSegs_dorder类中的同名函数返回值是不一样的，在那个类中，findUnusedPnt函数的返回值为点在列表中的序号（整数）。
- **findNextPnt**函数的作用是根据当前点从dic中查询下一个可以被直接添加到多段线poly中的点。
  - 这个函数看似简单，只有寥寥数行，但事实上，它是真正体现字典查询拼接算法精妙的地方。
  - 它的精妙之处就是通过字典结构，将两个坐标重合的点绑定在一起，从其中一个点能很快查询到另一个点。
  - 在循环中，findNextPnt起到了承上启下的作用，正因为这个函数的巧妙设计，才能够让循环以最简单的形式不断运转。

4.字典查询法拼接优化

- **全局函数**

- 在SliceAlgo模块中创建一个接口函数，函数名为**linkSegs_look**，用它封装LinkSegs_dlook类中的算法，输入为截交线段列表segs，输出为封闭轮廓列表contours，具体代码如下：

SliceAlgo.py	linkSegs_dlook：添加基于字典查询的截交线段拼接算法	6-8

```
1  from LinkSegs_dlook import *          # 导入LinSegs_dlook模块
2  ……                                    # 忽略SliceAlgo其他导入模块及代码
3  def linkSegs_dlook(segs):                    # 定义函数
4      return LinkSegs_dlook(segs).contours     # 返回封闭contours
```

5.算法测试比较

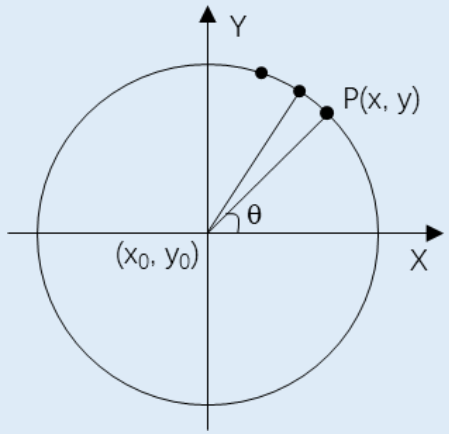
- **三种散乱线段拼接算法**
  - 暴力拼接法 $O(n^2)$
  - 字典序排序拼接法 $O(nlog(n))$
  - 字典查询拼接法 $O(n)$
- 后两种算法优于暴力法，但具体快多少，字典序排序法和字典查询法比孰优孰劣，都需要实际测试比较。
- 测试需要大量数据，即大量平面**散乱线段**，如何获取测试数据集？
  - 在平面上呈散乱无序的分布状态，
  - 内在逻辑上能连接成若干封闭轮廓，
  - 在数量上至少能够达到百万甚至千万级别。

5.算法测试比较

- **散乱线段生成** **无序性、连接性、海量性**
- 不能妄想随机生成的一些线段恰好能够连成轮廓。转变思路，**从先有轮廓的角度出发**，生成一些封闭轮廓，然后对轮廓细分，得到线段，打乱线段顺序。
- 选择一系列**同心圆**作为目标轮廓（为什么不用三角形、四边形等？）

如果圆心坐标为(x_0, y_0)，半径为r，那么圆上的任意点(x, y)可以表示为：

$$\begin{cases} x = x_0 + r \cdot \cos(\theta) \\ y = y_0 + r \cdot \sin(\theta) \end{cases}, \qquad (6\text{-}1)$$



- 对θ在其取值范围内细分，在每个分度上可以得到一个点，然后将这些点依次连接，就可以得到一系列线段。
- 对平面上所有同心圆，采取一样的细分操作，最后可以得到一个保存了所有同心圆上所有细分线段的线段列表，记为segs。

5.算法测试比较

- **散乱线段生成**
- 列表segs满足了以上提的两个要求：
  - 一是segs中线段的数量可由对每个圆细分的份数和同心圆个数来控制，达到百万甚至千万级别完全不是问题；
  - 二是segs中各线段在内在逻辑上也能连成若干封闭轮廓。
- 但是segs目前还不满足线段散乱分布的要求。
- 如何打乱segs中的各个线段？
  - 打乱segs必然需要用到随机数；
  - 遍历segs中每个元素，让当前序号的元素和一个序号随机生成的元素交换。

5.算法测试比较

- **散乱线段生成**
- 结合以上思路，实现散乱线段的生成函数**makeMessySegs：**

MakeMessySeg.py | **makeMessySegs函数：生成平面上大规模的散乱线段** | **6-9**

```
1   from Segment import *                                   # 导入Segment模块
2   from random import *                                    # 导入随机数生成模块
3   import math                                             # 导入math模块
4   def makeMessySegs(circleNum = 10, segNumPerCircle = 100000, radius = 100.0): # 函数定义
5       segs = []                                           # 定义线段列表
6       r = radius                                          # 以半径递增的方式生成同心圆
7       for i in range(circleNum):                          # 遍历，生成指定cirlceNum个同心圆
8           pnts = []                                       # 点列表，用于收集圆上的点
9           for j in range(segNumPerCircle):                # 按细分份数细分圆
10              theta = j / segNumPerCircle * 2 * math.pi   # 角度theta
11              x = r * math.cos(theta)                     # 圆上点x、y坐标，圆心在原点
12              y = r * math.sin(theta)
13              pnt = Point3D(x, y)
14              pnts.append(pnt)                            # 将点添加到pnts
15          pnts.append(pnts[0])                            # 在pnts添加起点，以封闭轮廓
16          for j in range(len(pnts) - 1):                  # 将点依次整理为线段，添加至segs
17              seg = Segment(pnts[j], pnts[j+1])
18              segs.append(seg)
19          r += 10.0                                       # 同心圆半径递增
20      print('segment count:', len(segs))                  # 打印segs中线段总数
21      print('min segment length:', segs[0].A.distance(segs[0].B))  # 打印segs中最短的线段
22      for i in range(len(segs)):                          # 对segs中线段散乱化处理
23          rd = randint(0, len(segs) - 1)                  # 生成一个随机序号
24          segs[i], segs[rd] = segs[rd], segs[i]           # 当前线段和随机序号线段交换
25      return segs
```

5.算法测试比较

- **散乱线段生成**

- 以上代码需要用到正余弦函数，因此需要导入**math**库；又因为要用到随机数生成函数，因此需要导入**random**库。这两个库都是Python的内建基础库，不需要额外安装。

- 函数**makeMessySegs**有3个参数，其中：
  - circleNum用于指定生成的轮廓数量，默认为10；
  - segNumPerCircle用于指定对每个圆的细分份数，默认为100000份，即每个圆最终可得到100000条线段；
  - radius用于指定同心圆的最小半径，默认为100mm，其余圆半径以10mm递增。

- makeMessySegs函数打印了线段列表中**最短的线段长度**，这是为什么？为什么要知道最短线段长度？

5.算法测试比较

- **拼接算法测试**
- 使用makeMessySegs生成的散乱线段测试3种线段拼接算法，测试用主机CPU为intel Core i7-8700 @ 3.20GHz CPU。

Test_linkSegs.py　测试3种拼接算法的性能　6-10

```
1   from SliceAlgo import *                 # 导入SliceAlgo库
2   from MakeMessySegs import *             # 导入散乱线段生成库
3   segs = makeMessySegs(10, 10000)         # 生成散乱线段，保存到segs
4   start = time.clock()
5   contours1 = linkSegs_dorder(segs)       # 1. 字典序排序法拼接，dorder，并计时
6   end = time.clock()
7   print("dorder: %f CPU seconds" % (end - start))
8   start = time.clock()
9   contours2 = linkSegs_dlook(segs)        # 2. 字典查询法拼接，dlook，并计时
10  end = time.clock()
11  print("dlook: %f CPU seconds" % (end - start))
12  start = time.clock()
13  contours3 = linkSegs_brutal(segs)       # 3. 暴力法拼接，brutal，并计时
14  end = time.clock()
15  print("brutal: %f CPU seconds" % (end - start))
```

5.算法测试比较

• 拼接算法测试

- 对makeMessySegs函数，将第1个参数设置为10且保持不变，即轮廓数为10；对第2个参数，在每轮测试中分别设置为100、200、500、…、10^6，最终得到的对应散乱线段数量分别为1000、2000、5000、…、10^7。

三种线段拼接算法运行时间统计与比较

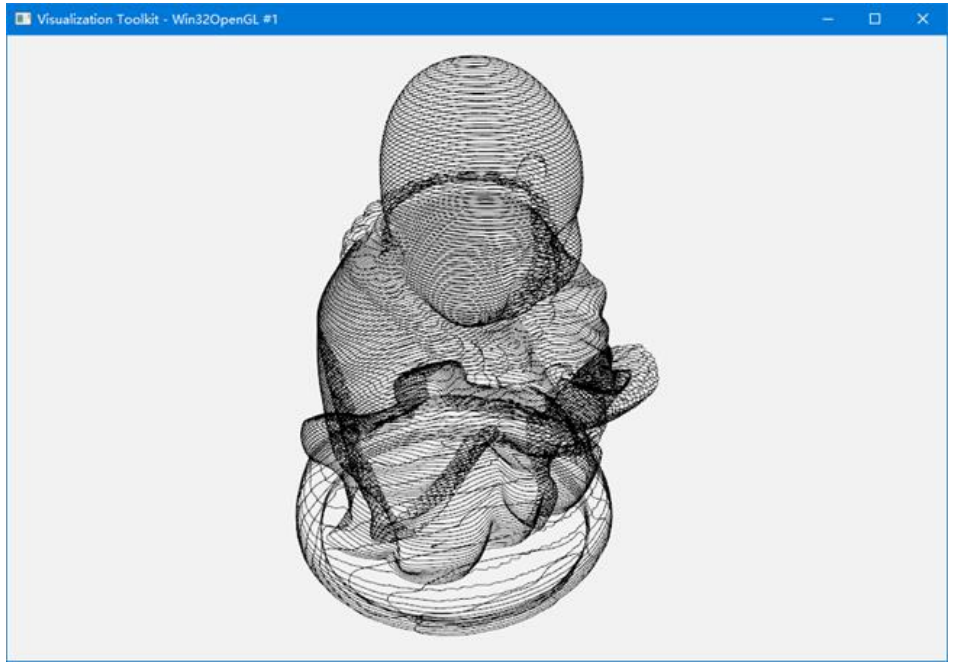
时间(秒) 线段数 / 方法	1,000	2,000	5,000	10^4	10^5	10^6	5×10^6	10^7
字典序 dorder	0.018	0.029	0.077	0.17	2.12	24.16	135.62	277.66
字典查询 dlook	0.008	0.017	0.054	0.1	1.16	13.45	67.22	133.95
暴力 brutal	1.06	4.31	27.21	110.02	…	…	…	…

• 算法性能结论

- 字典序排序（dorder）和字典查询法（dlook）的性能明显优于暴力法（brutal）。当线段数量为1000时，前两者性能是暴力法的100倍左右。
- 字典序排序和字典查询法两者性能基本处于同一水平线上，但**字典查询法明显更胜一筹**，它的性能大概是字典序排序法的2倍

5.算法测试比较

- **拼接算法测试：实际三维打印截交线段拼接测试**
- monk模型，面片数约为33万，高度为175.5mm。
- 切片厚度为0.2mm，得到的切片总层数为877层，截交线段总条数接近87万条，平均每层线段数约988条。
- 对得到的截交线段，使用暴力法拼接的时间约为975秒，超过16分钟；使用字典序排序法拼接的时间约为13.1秒，使用字典查询法拼接的时间约为8.8秒。



本章小结

- 这一章研究了三维打印路径规划中的截交线段拼接问题，实现了两种的线段拼接算法，即：字典序排序拼接法和字典查询拼接法。
  - 前者依赖对线段端点进行字典序排序，从点的序号中找出相邻的重合点；
  - 后者以线段端点坐标为键建立字典结构，从字典的键值对中查询重合点。
- 两种方法的相同点是它们都依赖**链接点（LinkPoint）**数据结构。
- 两种拼接算法具有线性的运行时间，它们的性能要远远优于暴力拼接法。就两种算法本身而言，字典查询法的性能比字典序排序法的性能更胜一筹。





思考题

1. 为什么LinkPoint数据结构既能表示一个点，又能表示一条线段?
2. 在三维空间中，对点进行字典序排序的几何意义是什么?
3. 说明基于点的字典序排序法能够提升平面线段拼接效率的本质原因?
4. 如果STL模型的三角面片之间存在缝隙，如何在拼接过程中得到封闭轮廓?
5. 简述字典查询法线段拼接优化算法的原理、过程及效率提升的原因。
6. 在字典查询法线段拼接优化算法中，为什么使用点的坐标元组而非点本身作为字典的键?
7. 除字典排序法和字典查询法外，尝试设计一种新的线段快速拼接算法。